

Documentação Trabalho Prático 1 - Redes

Professor: Filipe Vieira

Aluno: Mateus Faria Zaparoli Monteiro - RM: 2023028560

1. Desafios encontrados

Os principais desafios encontrados para a realização deste trabalho foram o entendimento de como usar na prática os sockets e de como implementar as conexões entre servidores e clientes.

Entender a teoria não foi difícil porém pelo fato de ser a primeira vez que estava vendo a implementação de um socket, saber o retorno das funções causou estranhamento, porém esse estranhamento não se justifica pois o uso as funções de criação do socket, de bind, de listen, de accept, de send e de receive é bem lógico, o retorno simples e a implementação é simples. Depois do entendimento de como funcionam essas tecnologias o próximo desafio encontrado foi a implementação da lógica do jogo, para isso pensei em algumas estratégias, mas acabei seguindo quase que a ordem das regras descritas no enunciado do trabalho.

Essa estratégia me pareceu um pouco verbosa mas rendeu bons frutos para tratar bugs e comportamentos inesperados ao longo do trabalho, fazendo com que o resultado do trabalho ficasse corretamente de acordo com a lógica pedida no enunciado.

Por fim, pensando qual foi o maior desafio, cheguei à conclusão que realmente entender o uso de sockets e estabelecer uma primeira conexão que funcione entre cliente e servidor foi o maior desafio e também o maior aprendizado adquirido com o trabalho.

2. Estratégias adotadas

Para a realização deste trabalho prático a primeira estratégia foi entender como um todo qual era o resultado final esperado para o trabalho, entendendo primeiro os requisitos funcionais e como o jogo citado deveria se comportar no final para depois se preocupar com requisitos não funcionais, uma vez entendido isso passei passo a passo a entender os requisitos não funcionais relacionados, primeiramente à parte de redes, sockets e portas, e depois à lógica do jogo propriamente dito.

Desse modo o primeiro passo dos requisitos não funcionais foi entender como passar os conceitos e teorias aprendidas sobre o uso de sockets e do IPv4 e IPv6 para a prática e como usá-los em uma implementação real criando o socket e chamando as funções necessárias para estabelecer conexão, ainda que local, entre um servidor e um cliente.

Assim, com base na imagem de estabelecimento de conexão com confiança mostrada em sala, em que se tinha as etapas tanto do cliente como do servidor, passei a utilizar as funções mostradas nos slides disponibilizados e na playlist do Prof. Ítalo tais como `socket()`, `bind()`, `listen()`, `accept()`, `recv()`, `send()` e `close`, para

implementar primeiramente o arquivo responsável pelo servidor, implementando e tratando alguns possíveis erros de memória e de utilização.

Primeiramente, na implementação inicial coloquei printf's de depuração no próprio server para saber se cada etapa do processo estava funcionando conforme esperado após cada função chamada, e até algumas propositalmente erradas para saber o formato de erro, o que futuramente eu retirei para não poluir o terminal que estiver rodando o servidor, a fim de deixá-lo limpo e de tal forma que ele só exibe mensagem caso aconteçam erros de fato e principalmente erros do usuário.

Finalizada a implementação do arquivo do servidor, passei então a implementar o arquivo do cliente, primeiro criando o socket() e tentando uma conexão com o servidor, para isso implementei no servidor a troca de mensagens inicial, de forma que eu saberia se a conexão estava funcionando para IPv4 e IPv6 e saber se podia prosseguir com a lógica do jogo, essa parte da primeira conexão entre server e cliente foi a mais crítica para o funcionamento do trabalho e a que necessitou de mais atenção e aprendizagem para ser desenvolvida, resolvida essa parte mais custosa e realmente nova do trabalho o restante do trabalho foi mais fácil conceitualmente porém mais demorada na escrita, isso se deve a ser uma parte mais repetitiva e após feita a primeira lógica do jogo, com o loop esperando e enviando mensagens, as demais partes do jogo foram basicamente entender o que o enunciado pedia para cada situação de jogo e criar as mensagens de resultado a serem implementadas.

Em relação à lógica do jogo em si, primeiro procurei entender se o menu de opções deveriam aparecer a cada interação com o jogo ou apenas no começo do jogo, a respeito do que entendi que o menu deveria aparecer a cada interação do cliente, depois, resolvi a lógica dos usos, por parte do cliente e do servidor, do Hyper Jump, acabando o jogo e printando o resultado final caso algum deles ou ambos os dois escolhessem a opção número 4. Após isso, fiz uma lógica de ifs para capturar e tratar cada possível opção escolhida pelo usuário e dentro de cada opção do usuário qual seria o resultado com cada possível escolha do servidor, e ao fim enviar uma mensagem de resultado de combate para o usuário, por fim, caso alguma nave chegasse ao hp 0 ou se alguma ou ambas as naves fugissem o código do servidor envia uma mensagem de Game Over terminando o jogo.

Do lado do cliente, implementei o loop do jogo que envia a mensagem contendo a opção escolhida pelo usuário e caso a mensagem que o servidor envia seja de game over o cliente sai do loop e fecha o socket.

O servidor, por sua vez, só fecha o socket e encerra o programa caso o programador manualmente o feche, caso contrário o servidor continua esperando novas conexões de clientes.

3. Possíveis melhorias

Como possíveis melhorias seria interessante implementar testes de unidade para o programa, e principalmente a implementação de uma GUI interativa com um menu inicial que possibilitasse que o cliente se conectar com o servidor utilizando o protocolo IP correto, e, após isso, uma tela de jogo com naves mostrando os

poderes que cada nave utiliza e efeitos sonoros para cada uma das opções do usuário.

Ademais, a inclusão de camadas de segurança quanto ao número de requisições e à funções que não deveriam ser expostas é uma questão que poderia ser incluída e em um sistema real de jogo seria necessário para garantir uma implementação segura para o servidor.

Concluí-se assim, que o uso de mais verificações de segurança e uma interface amigável ao usuário são pontos de possível melhoria para o programa e que seriam muito bem vindas em um programa que tivesse foco comercial.